

Relatório Técnico — Plataforma de Análise e Comparação de Apólices D&O

Projeto Final · Curso InsurMinds · Instituto de Inteligência Artificial Aplicada (I2A2)

Grupo Jarvis — Daniel Ramon, Nicole Paes, Paulo Henrique, Paulo Roberto e Juliana Catarina (representante)

Repositório: <<https://github.com/DanielRamon10/insurminds-projeto-final>>

1. Arquitetura da solução

A plataforma lê apólices de seguro D&O em PDF ou imagem, extrai as cláusulas que importam, guarda o resultado e compara duas ou mais apólices apontando o que as distingue — **sempre dizendo de que página de que documento cada informação saiu**.

```
`` documento (PDF ou imagem) | v INGESTAO (app/domain/ingestao.py, app/clients/extracao.py) le
pagina a pagina; OCR apenas onde falta texto embutido | v DocumentoExtraido – o texto nunca
perde o numero da pagina | EXTRACAO (app/agents/extrator.py, llm.py, validacao.py) o LLM
procura os 15 campos de data/campos_do.yaml | v ApoliceExtraida – cada campo guarda pagina e
trecho de origem | ARMAZENAMENTO (app/domain/armazenamento.py) SQLite; a rastreabilidade
sobrevive a ida e volta | v COMPARACAO (app/domain/comparacao.py) campo a campo, SEM LLM:
deterministico e testavel | v Comparacao – igual | diferente | redacao divergente | ausente |
REDACAO (app/agents/redator.py) INTERFACE (app/interface/) explica em prosa o que difere tela
com rastreabilidade clicavel ``
```

A decisão que atravessa todas as camadas: **o texto nunca perde o número da página, e todo campo extraído guarda o trecho de onde saiu**. É o que separa uma extração auditável de um chute bem formatado — e num documento jurídico um limite de indenização inventado é pior que nenhum.

Separação entre o que é determinístico e o que é generativo

Etapa	Usa LLM?	Por quê
Ingestão	não	leitura de arquivo é problema resolvido
Extração das cláusulas	sim	achar uma cláusula sob outro nome exige leitura
Comparação	não	decidir <i>o que</i> difere precisa ser reproduzível e testável
Redação do relatório	sim	explicar <i>por que</i> a diferença importa exige linguagem

Essa fronteira é o que permite **259 testes automatizados sem tocar a rede nem gastar cota**: tudo o que é decisão de negócio é determinístico.

2. Tecnologias utilizadas

Camada	Tecnologia	Papel
Leitura de PDF	<code>pypdf</code>	extrai texto embutido, preservando o número da página
Renderização	<code>pypdfium2</code>	converte página em imagem para o OCR, sem depender do poppler
OCR	Tesseract + <code>pytesseract</code>	lê documento escaneado e imagem, em português
Contratos de dados	Pydantic v2	valida a fronteira entre as frentes de trabalho
Regras de negócio	YAML	dicionário de campos alterável sem tocar em código
Modelos de linguagem	LangChain + Google Gemini, Groq, OpenRouter	extração e redação
Armazenamento	SQLite	guarda as extrações com a rastreabilidade intacta
Interface	Streamlit	demonstração da comparação
Apresentação	python-pptx	pitch deck gerado por script, com números vindos do código
Testes	pytest	259 testes, nenhum dependente de rede

Linguagem: Python 3.10+. **Volume:** cerca de 7.000 linhas de aplicação e scripts, e 3.260 de testes.

3. Descrição dos agentes desenvolvidos

3.1 Agente de ingestão

Transforma o arquivo recebido em texto página a página. Decide **por página, e não por documento**, entre usar o texto embutido no PDF e recorrer ao OCR — um PDF nativo com um anexo escaneado no meio perderia justamente essa página se a escolha fosse feita uma única vez para o arquivo inteiro.

Recusa formatos que não sabe ler dizendo o que aceita, e isola falhas: um documento corrompido no meio de um lote não impede o processamento dos demais.

3.2 Agente extrator

Recebe o texto com marcadores de página e o dicionário de campos, e devolve os campos estruturados. Usa cascata entre provedores de LLM: se a cota de um esgota, o seguinte assume. **Uma apólice consome cerca de 60 mil tokens de entrada**, o que torna a cascata necessária e não precaução.

Um campo que a apólice não trata recebe `null` explícito, nunca string vazia — a distinção entre "não encontrei" e "não existe" é o que permite à comparação destacar a ausência.

3.3 Agente validador

Confere a saída do extrator antes de ela virar dado:

- **Número sem apoio no trecho citado** é sinalizado — cada número do valor precisa

aparecer na citação de origem.

- **Trecho que não existe no documento** invalida a origem.
- **Trecho repetido em mais de três páginas** não serve como localização: é cabeçalho

ou rodapé, não prova de lugar.

- **Remissão disfarçada de valor** — quando o modelo devolve "conforme a Especificação

da Apólice" como se fosse conteúdo, o validador converte para `null`.

3.4 Agente redator

Escreve em prosa o que as diferenças significam. **Só afirma que duas cláusulas divergem quando consegue citar o trecho das duas apólices sobre o mesmo ponto**; sem as duas provas, marca o campo como inconclusivo e recomenda a leitura.

Na comparação entre as duas apólices reais, produziu 2 diferenças afirmadas, 1 equivalência e 6 inconclusivos. As quatro citações do relatório foram conferidas uma a uma contra os PDFs: **todas existem literalmente**.

3.5 Motor de comparação (determinístico, sem LLM)

Percorre o dicionário do especialista — e não os campos que cada apólice trouxe — para que a ausência apareça como ausência. Compara por tipo, não por string: `R$ 10.000.000,00` e `R$ 10 milhões` são o mesmo limite escrito de dois jeitos, assim como `90 dias` e `3 meses`. Para texto corrido, **recusa-se a julgar**: devolve "redação divergente" e deixa a leitura para o redator, em vez de afirmar uma diferença que não verificou.

3.6 Interface

Apresenta a comparação com a origem de cada valor acessível em um clique, e renderiza a página do PDF ao lado do trecho citado. Exporta a comparação em CSV com separador `;` e BOM, para abrir no Excel em português. Toda extração de exemplo aparece marcada como tal em três lugares da tela.

4. Fluxo completo de processamento

1. **Recepção** — o documento chega em PDF ou imagem; formatos desconhecidos são

recusados com mensagem que diz o que a plataforma aceita.

1. **Extração do texto** — página a página, preservando a numeração. Páginas sem texto embutido são renderizadas como imagem e passadas pelo OCR.

1. **Montagem do contexto** — o texto é entregue ao modelo com `=== PÁGINA n ===` antes de cada bloco, para que ele possa citar a página e para que a citação seja conferível.

1. **Extração das cláusulas** — o modelo procura os 15 campos do dicionário, usando os 46 sinônimos registrados, porque a mesma cláusula aparece com nome diferente em cada seguradora.

1. **Validação** — números sem apoio, trechos inexistentes, localizações falsas e remissões disfarçadas de valor são tratados antes de o dado ser aceito.

1. **Armazenamento** — a apólice é gravada com a rastreabilidade intacta. Reprocessar substitui a extração anterior em vez de acumular versões.

1. **Comparação** — campo a campo, sem LLM, classificando cada um entre igual, diferente, redação divergente, ausente em uma das apólices ou ausente em todas.

1. **Redação** — o relatório em prosa explica as diferenças que podem ser provadas, e declara inconclusivas as que não podem.

1. **Apresentação** — a interface agrupa por tipo de diferença, mostra os valores lado a lado com a explicação do especialista, e abre a origem de qualquer valor.

5. Justificativa das decisões arquiteturais

O dicionário de campos é do corretor, não do programador. `data/campos_do.yaml` define os 15 campos, seus 46 sinônimos e, para cada um, a frase que explica ao usuário por que aquela diferença importa. Foi preenchido pelo corretor de seguros do grupo, e muda sem tocar em código. Três dos campos — sublimites, definição de reclamação e cobertura para investigações — foram acrescentados por ele e não estavam na lista inicial. O mais valioso é *sublimites*, com a justificativa dele: duas apólices podem ter o mesmo limite máximo e coberturas específicas diferentes.

A comparação não usa modelo de linguagem. Decidir o que difere precisa ser reproduzível: a mesma entrada produz sempre a mesma saída, e o comportamento inteiro é testável sem rede. Explicar por que a diferença pesa é outra tarefa, e fica com o redator.

O motor compara por tipo e se recusa a julgar texto. Comparar `R$ 10.000.000,00` com `R$ 10 milhões` por igualdade de string produziria o falso positivo mais grosseiro possível, e as condições gerais

alternam entre as duas formas. Já para cláusulas em texto corrido, afirmar divergência sem ter lido seria inventar.

O redator precisa de duas provas para afirmar uma diferença. Num documento jurídico, apontar divergência inexistente custa mais que admitir dúvida.

SQLite, sem ORM. O enunciado pede armazenamento estruturado; um MVP que exige subir um serviço de banco para ser demonstrado é um MVP pior. São duas tabelas e meia dúzia de consultas. Há banco, e não apenas processamento em memória, porque reextrair uma apólice custa ~60 mil tokens e porque guardar só o valor perderia a prova de origem.

Nenhum dado fabricado sem aviso. As extrações de exemplo, usadas quando o banco está vazio, aparecem marcadas na tela em três lugares. O custo de dizê-lo é uma linha de texto; o de omiti-lo seria a credibilidade do projeto.

Cascata entre provedores de LLM. Uma comparação entre duas apólices custa cerca de 120 mil tokens de entrada, e o desenvolvimento repete isso dezenas de vezes. A ordem da cascata é por qualidade, não por disponibilidade: extrair cláusula jurídica é tarefa em que um modelo fraco inventa número.

Credenciais fora do código. As chaves vêm do `.env`, que está no `.gitignore`. O script de empacotamento varre o conteúdo dos arquivos e **se recusa a gerar o pacote** se encontrar qualquer credencial.

6. Limitações conhecidas

Os campos numéricos não vêm das condições gerais. Limite máximo de indenização, franquia, vigência, retroatividade e sublimites saem `null` nas duas apólices processadas. Não é falha da extração: condições gerais são o documento-modelo da seguradora, e esses valores pertencem à **Especificação da Apólice**, que é individual de cada contratante e não é pública. A plataforma registra a cláusula e sua página, anotando que o valor está em outro documento.

O redator trabalha sobre recortes. Ele recebe o resumo que o extrator produziu, e não a cláusula inteira. Com recorte parcial, há casos em que um corretor afirmaria uma diferença e o redator responde inconclusivo — foi o que aconteceu em 6 dos 9 campos comparáveis. A escolha é deliberada: preferir o silêncio à invenção.

Dois apólices, uma linha de produto. O dicionário foi construído para D&O. Aplicá-lo a outro ramo exigiria refazer os campos com o especialista — o que é barato, por serem um arquivo YAML, mas não é automático.

O OCR existe, mas não foi exercitado em escala. As apólices públicas reunidas são PDFs nativos. O caminho de OCR foi testado renderizando páginas reais como imagem e relendo-as, mas nenhum documento escaneado de verdade passou pela plataforma.

Sem avaliação quantitativa da extração. Conferimos que todas as 19 citações produzidas existem literalmente nos PDFs, com a página correta. Não medimos, porém, quantas cláusulas presentes nos

documentos o extrator **deixou de encontrar** — isso exigiria uma marcação manual dos dois documentos por um especialista.

7. Possibilidades de evolução futura

Dar ao redator a cláusula inteira. A causa dos inconclusivos é o recorte parcial. Entregar ao redator o texto completo da cláusula — ou a página de origem — daria base para afirmar diferenças que hoje ficam em aberto, sem relaxar a exigência de prova.

Aceitar a Especificação da Apólice. Com o documento individual, os campos numéricos passam a ser extraídos e a comparação ganha a dimensão de valores. A arquitetura já suporta: é um documento a mais pela mesma ingestão.

Medir a cobertura da extração. Marcar manualmente as cláusulas de duas apólices e comparar com o que o extrator encontra permitiria dizer quantos por cento ele acerta — hoje sabemos que não inventa, mas não quanto deixa passar.

Ampliar o dicionário com o especialista. Quinze campos cobrem a comparação essencial; um corretor usa mais em negociações complexas. Como o arquivo é YAML, o custo de crescer é de conversa, não de desenvolvimento.

Comparar mais de duas apólices na tela. O motor já aceita N apólices; a interface mostra duas.

Histórico de versões da apólice. Guardar extrações sucessivas do mesmo documento permitiria acompanhar o que mudou entre renovações — pergunta frequente de quem administra carteiras.

Anexo — números da execução

Apólices processadas	2 (Chubb e AIG), 70 e 72 páginas
Campos do dicionário	15, com 46 sinônimos
Campos extraídos	10 na Chubb, 9 na AIG
Campos rastreáveis	100% dos encontrados
Citações conferidas no PDF	19 de 19
Custo por apólice	~60 mil tokens de entrada
Modelo utilizado	<code>gemini-3.1-flash-lite</code> (cascata com Groq e OpenRouter)
Testes automatizados	259 , nenhum dependente de rede

As condições gerais utilizadas são documentos públicos, publicados pelas próprias seguradoras; as fontes estão citadas em `data/apolices/FONTES.md`. Nenhum documento contém dado de cliente.